Beta-Testing Monopoly: Closed-Loop Game-Design Optimisation over a Diverse Strategy Pool

Selim Emir Can (selimcan@stanford.edu) Alaz Cig (alaz@stanford.edu)

2026-06-06

Abstract

We present a closed-loop, agent-driven beta-testing workflow for game design, instantiated on Monopoly. A genetic algorithm searches a 66-dim board design space (per-property cost and rent multipliers and a 22-bit keep-mask) under a composite objective combining fairness, game length, draw rate, and inter-player money transfer, evaluated against a fixed pool of 30 parametric rule-based strategies. A second agent class, a 1.5B-parameter instruction-tuned LLM (Qwen2.5-1.5B) wrapped in a structured echo-validated prompt, re-evaluates the same boards as an independent cross-class probe. The rule-based GA improves the composite score by approximately 47% over the default board at both 2 and 3 players, outperforming a random-search baseline at matched evaluation budget by $\sim 13\%$ in both regimes. Single-objective ablations confirm that every term in the composite is non-redundant. A $2p \leftrightarrow 3p$ cross-evaluation at n=1000 shows that each design specialises to its training regime: the off-regime composite is 16−24% worse than the regime-matched optimum, though both designs still beat the default by a wide margin under either evaluator. The LLM probe reproduces the rule-based directional ranking with zero model-side hallucinations across 2,207 calls under structured validation. Re-running the GA with the LLM in the evaluator role yields a board that beats the default but loses to the rule-based winner under the diverse pool: the fairness gap is 0.20 vs. 0.379 at 2p and 0.05 vs. 0.404 at 3p, diagnosing the blind spots of any single-personality evaluator. Code, logs, and figures: https://github.com/Selim-Emir-Can/CS348K-proj/tree/master.

1 Background and setup

The problem. Game designers face combinatorial design spaces that human playtesting cannot cover. Tabletop Monopoly is a sharp testbed for the agent-assisted alternative: a small but rich parameter space and well-known balance issues (games drag on, single-property luck dominates, two-player play is unbalanced). The goal of this project is not to "solve" Monopoly but to demonstrate a reproducible designer-facing beta-testing loop:

> **Parameterise environment → evaluate against a diverse agent population → compare agreement across agent classes → shortlist candidate designs → send only the most informative cases to human playtest.**

Inputs and outputs. The pipeline takes a board configuration $\theta \in \mathbb{R}^{44} \times \{0,1\}^{22}$ (22 cost multipliers, 22 rent multipliers, 22 keep-mask bits) and produces (i) a scalar composite score plus a per-component metric tuple $(\overline{F}, F_{\max}, \overline{R}, \overline{D}, \overline{T})$ estimated over 100 games against the strategy pool, and (ii) a decision-quality artefact (per-strategy heatmap, per-game JSONL, or per-decision LLM log) that lets the designer audit *why* θ scored as it did.

Goals and constraints. The optimiser minimises a weighted sum of five design qualities (Eq. 1), chosen so that no single term dominates: mean pair-fairness $\overline{F}$, worst-pair fairness $F_{\max}$, length deviation $|\overline{R} - R^*|/R^*$, mean draw rate $\overline{D}$, and player-to-player money-transfer deviation $|\overline{T} - T^*|/T^*$. Hard constraints: every candidate runs end-to-end at the per-property cost/rent bounds $[0.5, 2.0]$, retains both utilities and railroads,

and finishes or truncates within 200 rounds. Soft constraints: total wall-clock budget under 30 minutes on a single CPU ($\approx$ 6 ms/game) for the rule-based pipeline, and under 6 hours on a single 12 GB GPU for the LLM probe.

$$\text{score}(\theta) \;=\; w_{\text{fair}}\overline{F} \;+\; w_{\text{fmax}}F_{\max} \;+\; w_{\text{len}}\frac{|\overline{R}-R^*|}{R^*} \;+\; w_{\text{draw}}\overline{D} \;+\; w_{\text{money}}\frac{|\overline{T}-T^*|}{T^*} \tag{1}$$

Defaults: $w = (1, 0.5, 0.5, 0.3, 0.3)$, $R^* = 60$ rounds, $T^* = 100$ \$/round.

Hypothesis. The falsifiable claim of the project is that a candidate board's directional ranking under a 30-strategy parametric pool matches its directional ranking under an independent agent class (a 1.5B-parameter LLM with a deterministic per-field state validator), at a per-decision hallucination rate below 1%. This sits between "a single trained agent ranks designs" (insufficient for fairness) and "agents predict human win-rates" (likely hopeless, and not what a designer needs); it is precisely the claim required to use agent feedback as a beta-testing surrogate.

The crux. The hard part of the project is neither the GA over a 66-dim board space nor the LLM plumbing; it is the question of *what evaluates a candidate board* in the absence of human playtesters. A single trained agent (RL or LLM) used as an oracle collapses "design quality" to one playstyle's quirks: a board that scores well against an aggressive builder may be obviously broken against a cash hoarder. The honest alternative is to evaluate against a *population* of strategies, but the strategy space we care about is continuous (17 knobs: 5 continuous, 4 boolean, an 8-bit colour-group mask), so the real question is *which strategies, and how many*. The project's load-bearing design choice is the discretisation: 10 hand-designed archetypes (*AggressiveBuilder, CashHoarder, Trader, RailroadKing, LowCostOnly, HighCostOnly, Balanced, Passive, Bully, RiskAverse*) chosen to span qualitatively different playstyles, plus 20 samples drawn once from the parametric space with a fixed seed to fill the gaps between them. Thirty strategies is small enough that 100 games per candidate over 200 candidates fits in $\sim$ 25 minutes on a single CPU, and large enough that worst-case fairness across the pool is a meaningful objective rather than a coin flip on one matchup. A secondary technical challenge, addressed as part of the cross-class validation rather than the central contribution, is making a small instruction-tuned LLM usable as an independent agent class without hallucinations leaking into the metrics; the solution is a structured `STATE/ECHO/REASON/ANSWER` prompt with deterministic per-field state validation and a bounded retry budget, reported with its failure modes (Section 4.5).

2 Related work

Search-based game design and automated playtesting. Procedural and search-based game-content generation has a long literature; Shaker et al. [8] survey the field, and prior tools such as Sentient Sketchbook [5] use search agents to surface design issues during authoring. Closest to our strategy pool is the line of work on *procedural personas* [3], which represents distinct player types as agents evolved or trained to maximise hand-designed utility functions, and uses them both to playtest and to drive content generation; evaluating or optimising a design against such an agent population is established prior art, not a contribution of this work. Our pool differs only in mechanism: fixed, hand-specified and sampled rule-based parameters rather than trained policies, since per-candidate model-free RL is impractical for a high-variance stochastic game evaluated over thousands of designs. The contribution is instead a protocol that uses *multiple agent classes* as cross-checks (a parametric rule-based pool and an independent LLM probe), with disagreement-as-diagnostic and human playtesting reserved for the boundary cases.

LLM-based agents. Small instruction-tuned LLMs such as Qwen2.5 [7] have been used as decision-making agents in board games, simulations, and tool-use evaluations; the line of work on generative agents [6] treats LLMs as a population of social actors. Our setup is narrower: a single 1.5B-parameter LLM with greedy decoding, a structured `STATE/ECHO/REASON/ANSWER` prompt, and a deterministic per-field validator with a bounded retry budget. The validator-and-retry stack is closer in spirit to constrained decoding than to standard chain-of-thought prompting; we report failure modes specific to that protocol (Section 4.5).

Tree search and competitive evaluation in games. Browne et al. [1] survey Monte Carlo tree search; population-based competitive evaluation has been used at scale for StarCraft II via leagues of agent variants [10]. We use the same "population as evaluator" intuition in inverted form: the population is fixed and diverse and the environment is what we search over.

Evolutionary algorithms over environment design. GAs are routine for environment-design problems; Goldberg [2] lays out the standard mechanisms (tournament selection, uniform crossover, Gaussian and bit-flip mutation, elitism) we use unchanged. The contribution lies in the *evaluator* (a 30-strategy parametric pool, with an LLM evaluator as a cross-class control), not in the search algorithm.

Robust evaluation. Variance reduction via shared random numbers is standard in stochastic simulation [4]; Wilson confidence intervals [11] are the standard way to quote per-cell uncertainty on win rates; per-objective single-knob ablations are standard in optimisation. We adopt all three because they are cheap, reproducible, and orthogonal. The novelty in this work is connecting them to the agent-as-design-probe framing.

Pipeline framework. The simulator harness builds on the PettingZoo multi-agent gym API [9]; our optimisation loop is a thin wrapper around the upstream Monopoly core game loop, instrumented to return per-game stats instead of only logging.

3 Approach

We started from a single-process Monopoly core game loop with a `Player` base class, a YAML round-trip `GameConfig`, and a small set of rule-based player subclasses, plus pretrained Qwen2.5-1.5B-Instruct [7] weights from Hugging Face. Every piece of the optimisation loop, the strategy pool, the design-space encoder, the per-game stat collection, and the LLM integration was new work for this project.

The optimisation stack lives under `monopoly/optimizer/` and `monopoly/scripts/`. `ParametricPlayer` together with `ParametricPlayerSettings` subsumes every existing rule-based subclass under a single 17-knob agent (5 continuous, 4 boolean, 8-bit colour-group mask). `strategy_pool.py` produces the 30-strategy pool described in Section 1, deterministic via a fixed seed and saved once for reuse across runs. `design_space.py` maps a 66-dim vector to a fresh `GameConfig` by applying per-property multipliers to `cost_base`, `cost_house`, `rent_base`, and `rent_house`, and physically removing properties whose keep-bit is zero (structurally shortening the lap rather than substituting `FreeParking`); the identity vector is the default board. `simulate.py` is a thin re-implementation of the inner game loop that returns a per-game stat dict (winner, rounds, bankruptcy map, inter-player transfer total) instead of only logging; inter-player cash flow is captured via a context-manager monkey-patch of `Player.pay_money`, with non-player payments (taxes, fines, salary) excluded, and trade calls are capped at 5 per (player, turn) to break an unbounded loop in upstream code. `search.py` implements both random search and a GA (population 20, 10 generations, tournament selection k=3, uniform crossover, σ=0.1 Gaussian mutation on continuous dims, 1-bit-flip mutation on the keep-mask, elitism 2); both produce identical JSONL.

The LLM layer is a single subclass `LLMPlayer` in `agents.py`: a pre-filter avoids LLM calls when affordability or cash floor decides the action, the prompt has the `STATE/ECHO/REASON/ANSWER` structure described in Section 1, and the validator parses the model's `ECHO` block with a multiline regex and equality-checks each of the 10 fields against ground-truth state (with a \$0.50 tolerance to absorb float drift). On any echo mismatch the player retries, up to `MAX_RETRIES=4`, replaying prior assistant turns plus a corrective message naming the mismatched fields. Every decision is logged via the `decision_log_path` kwarg; the log records the prompt, raw response, parse path, ms elapsed, generation metadata, echo mismatches, and the full attempt list.

Three design decisions did most of the work for the evaluation sections that follow. The validator-and-retry stack is treated as *part of the probe* rather than infrastructure around it, and we report both v1 and v2 versions because an LLM with a broken validator is a measurably different design probe than one with a working validator (Section 4.5). Shared random numbers across candidates make GA convergence visible at 100 games per candidate rather than the $\sim$ 1000 that uncorrelated evaluations would require: every candidate

is scored on the same 10 matchups × 10 seeds, and an unchanged design vector yields byte-identical metrics. Finally, the agent population is the evaluator, not the oracle: a single agent collapses design quality to its own quirks, while the 30-strategy pool exposes worst-case fairness as a first-class objective.

The end-to-end pipeline is driven by four scripts. `scripts/optimize_board.py` runs the rule-based search and emits one JSONL line per evaluation. `scripts/cross_eval.py` runs designs through both player-count harnesses at n=1000. `scripts/strategy_heatmap.py` produces 30×30 strategy-vs-strategy win-rate matrices on any chosen design. `scripts/eval_llm_on_boards.py` runs the all-LLM-seat probe with per-decision logging, and `scripts/optimize_board_llm.py` re-runs the GA with the LLM in the evaluator role.

4 Evaluation and results

Definition of success. The project is successful if all three of the following hold:

1. the GA finds a board that improves the composite score over the default by a margin larger than the noise floor, verified at the optimum with n=1000 and 95% Wilson intervals on every metric;
2. single-objective ablations show that each of the five composite terms drives a different optimum, establishing that the objective is non-redundant;
3. an independent agent class (LLM under structured ECHO validation) reproduces the same directional ranking on the optimised boards, at a per-decision hallucination rate below 1%.

A weaker fourth claim, treated as diagnostic rather than headline, is that running the GA with the LLM as evaluator should produce a board that behaves *plausibly* under the rule-based pool, even if it loses to the rule-based winner.

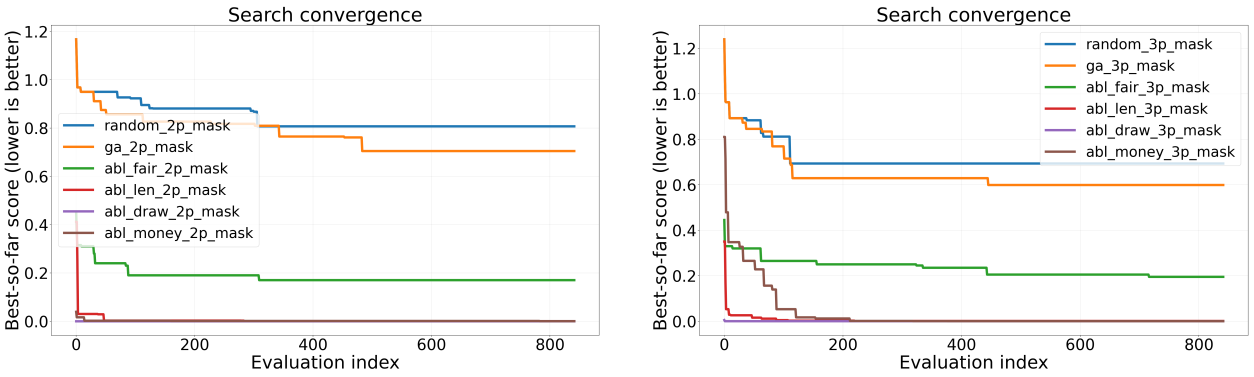
Experimental setup. The strategy pool is 30 parametric players (10 archetypes plus 20 sampled), pickled once with a fixed seed. For each player count $n \in \{2, 3\}$ we draw 10 evaluation matchups under diversity constraints: every archetype appears in at least one matchup, and at least 3 matchups include a sampled strategy. Each candidate is scored on $10 \times 10 = 100$ games with seeds shared across all candidates, so an unchanged design vector yields byte-identical metrics. Hardware: a single CPU at ≈ 6 ms/game for the rule-based stack; a single 11 GB GPU locally for LLM games at ≈ 6–10 s/decision. Total games for the entire study: ≈ 165k for the rule-based pipeline, ≈ 2300 for the LLM cross-class probe, and ≈ 2000 for the LLM-driven GA and its cross-evaluation.

4.1 Default-board baseline and search

The default board is the identity vector under the encoder. It scores 1.463 on the composite at 2p and 1.230 at 3p (full metrics in the `identity_default` rows of `logs/optimizer_v2/cross_eval_mask.json`); these are the reference points every optimisation result is compared against, with Wilson 95% CIs of ±3pp on individual win rates at the n=1000 deployment evaluation.

The GA outperforms random search consistently across the run, but not by an order of magnitude (Figure 1). The larger lift comes from problem framing rather than optimiser cleverness: the composite objective, the fixed strategy pool, and shared random numbers across candidates make random search on this problem already a strong baseline. This is consistent with a more general observation about system-design problems, where the bottleneck is usually problem formulation rather than search.

What does the GA actually build? Figure 2 renders the combined-objective GA winners alongside the default board for each player count. The 2p winner shrinks the lap from 40 cells to 30 (10 properties removed by the keep-mask), retains both railroads and utilities, and rebalances the remaining colour-group costs and rents. The 3p winner shrinks differently and keeps a different subset of properties, reflecting that a 3-player game pressure-tests a different strategy distribution and converges to a different physical board.



(a) 2 players

(b) 3 players

Figure 1: **The GA outperforms random search at matched budget, by $\sim$ 13% in both player-count regimes.** Best-so-far composite score (lower is better) over the candidate-evaluation budget, six runs per player count: random search (842 iters), the combined-objective GA (population 30×30 generations, 842 evaluations after 2-elitism), and the four single-objective ablations. At 2p the GA finishes at 0.705 vs random's 0.807 (-12.7%); at 3p the GA finishes at 0.599 vs random's 0.694 (-13.6%). Per-candidate evaluation uses 20 games per matchup (n_{games}=200) to halve variance relative to the n_{games}=100 default. The ablation runs reach much lower scores because each minimises only one term of the composite, previewing the non-redundancy claim made by Figure 3. The overall lift over the default board ($\sim$ 0.65 absolute, $\sim$ 47% relative) is several times larger than the GA-vs-random gap, which positions the bigger methodological lesson cleanly: problem framing dominates, optimiser choice contributes a smaller-but-real margin on top.

4.2 Single-objective ablations

We ran five ablations per player count, each setting one weight to 1 and the others to 0 (*fairness-only, length-only, draw-only, money-only, combined*), and reported best-so-far convergence and the full metric tuple at the optimum. Each ablation reaches a different optimum: the length-only run drives $\overline{R}$ to within ≈ 1 round of $R^* = 60$ but degrades fairness, while the fairness-only run nearly halves $\overline{F}$ but inflates draws. The per-aspect heatmaps in Figure 3 show the ablations carving visibly different shapes in the cost-multiplier, rent-multiplier, and keep-mask sub-spaces. The composite is therefore non-redundant: if any two terms drove the optimiser toward the same board, the corresponding ablations would have near-identical metrics, which they do not.

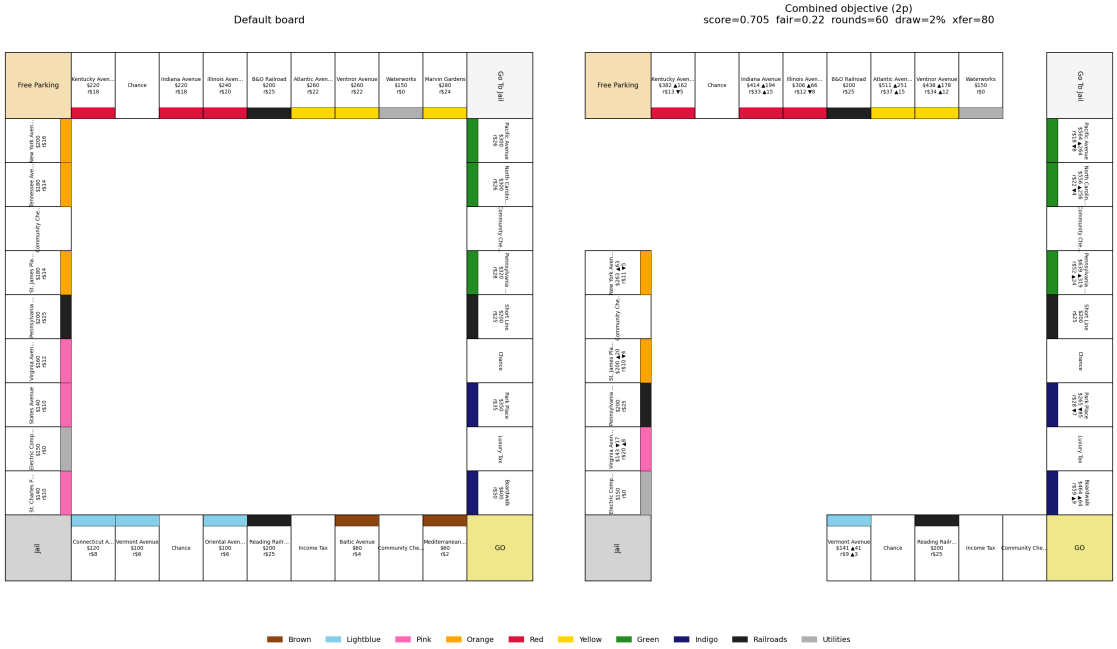
The same finding is visible at the level of physical boards (Figure 4): the five winners (combined + four ablations) are five visibly different shapes, with different properties kept or dropped and different costs/rents.

4.3 $2p \leftrightarrow 3p$ cross-evaluation

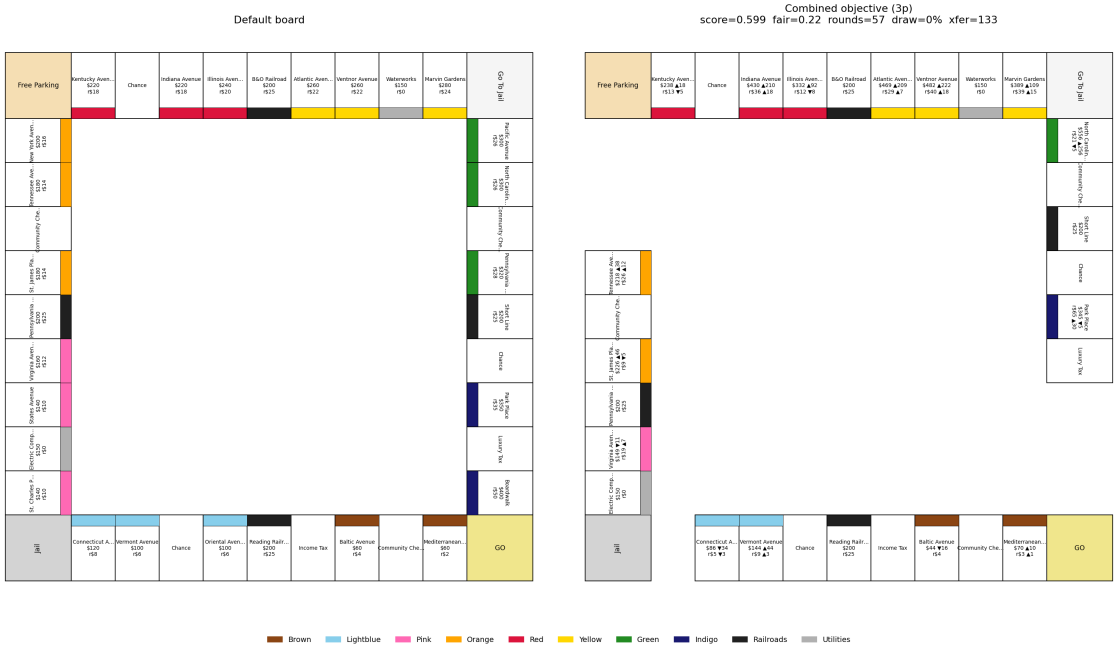
We took the combined-objective winner from each player-count regime and re-evaluated it under the other harness at n=1000 (Table 1, Figure 5). Two findings carry weight beyond the headline improvement over the default.

The first finding is that *each design specialises to its training regime, but both generalise across regimes without catastrophic failure.* The GA-2p winner improves the composite by 47% on its training regime ($1.463 \to 0.773$ at 2p) and is the best 2p design in the table; the GA-3p winner improves by 48% on its training regime ($1.229 \to 0.641$ at 3p) and is the best 3p design. Off-regime, each is still much better than the default but no longer the best: the GA-2p winner under 3p eval scores 0.795 (vs. GA-3p's 0.641, a 24% relative gap), and the GA-3p winner under 2p eval scores 0.897 (vs. GA-2p's 0.773, a 16% gap). The cleanest reading is that the GA finds a *training- regime-specialised* optimum: matching the design to the deployment regime is worth $\sim 16-24\%$ on the composite, but either design still beats the default by a wide margin under the off-regime evaluator, so a single design from either training regime could ship to either player count without disaster.

The second finding is that *optimisation-set overfit is small on both designs.* At the n=200 optimisation



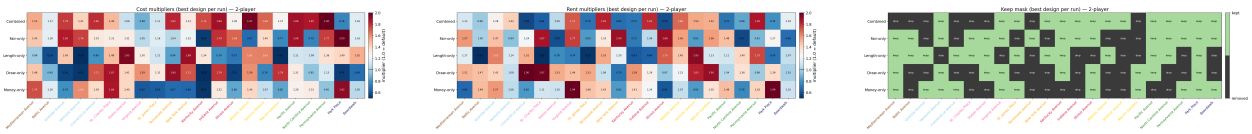
(a) 2-player combined-objective GA winner.



(b) 3-player combined-objective GA winner.

Figure 2: **The boards the GA actually builds.** Each panel shows the default Monopoly board (left, 40 cells) alongside the GA winner (right, shrunk by the keep-mask), with per-property cost/rent multipliers and keep/drop annotations on the optimised side. The 2p and 3p winners are visibly different physical boards, consistent with the cross-evaluation finding (Section 4.3) that each design specialises to its training regime.

budget the GA-2p winner scored 0.705 on its training matchup set; at n=1000 deployment it scores 0.773, an overfit of +0.068 (Table 2). The GA-3p winner scored 0.599 at training and 0.641 at deployment, an overfit of +0.042. Both gaps are small ($\leq 9\%$ of the deployment score) relative to the headline $\sim 47\%$ improvement



(a) cost mult. (2p) (b) rent mult. (2p) (c) keep-mask (2p)

Figure 3: **Each ablation carves a different shape in design space (2p).** Per-property cost multiplier (a), rent multiplier (b), and keep-mask bit (c) at the optimum of each GA run. Within each panel, *rows* are the five ablations (*combined, fair-only, length-only, draw-only, money-only*, top to bottom) and *columns* are the 22 ownable properties in board order (Mediterranean Avenue at left through Boardwalk at right). Cell values are the multiplier at the optimum (cost/rent panels, 0.5 to 2.0, with 1.0 being the default identity) or the keep-bit (keep-mask panel: green = kept, black = dropped). If any two ablations drove the optimiser to the same board, those rows would be near-identical; the visible inter-row variation is the empirical evidence that the composite objective is non-redundant.

Design	Eval	score ↓	$\overline{F}$ ↓	$F_{\max}$ ↓	rounds (→60)	draw % ↓	transfer (→100)
default board	2p	1.463	0.454	0.940	103.9	7.4	49.7
default board	3p	1.229	0.411	0.680	110.8	17.6	99.4
GA-2p winner	2p	**0.773**	**0.215**	**0.910**	62.2	1.7	73.3
GA-2p winner	3p	0.795	**0.273**	0.730	65.7	3.3	133.1
GA-3p winner	2p	0.897	0.287	0.970	56.2	**0.5**	69.4
GA-3p winner	3p	**0.641**	0.280	**0.490**	56.3	**0.6**	127.8

Table 1: **Cross-player-count evaluation at n=1000 games per cell, 66-dim keep-mask design space** (source: `logs/optimizer_v3/cross_eval_mask.json`). Wilson 95% CIs on individual win rates are ±3pp. Header arrows: ↓ means lower is better; (→T) means the target value is T and deviation in either direction is penalised. Bold cells mark the best (lowest) score and best mean fairness $\overline{F}$ within each evaluation regime. Each design is best on its own training regime: GA-2p winner at 2p (0.773 vs 0.897), GA-3p winner at 3p (0.641 vs 0.795). Both designs improve over the default by approximately 47% in their training regime (1.463 → 0.773 at 2p, 1.229 → 0.641 at 3p) and generalise to the off-regime within ∼ 16% on the composite without catastrophic failure.

over the default, so the directional improvement is robust to re-evaluation at higher n.

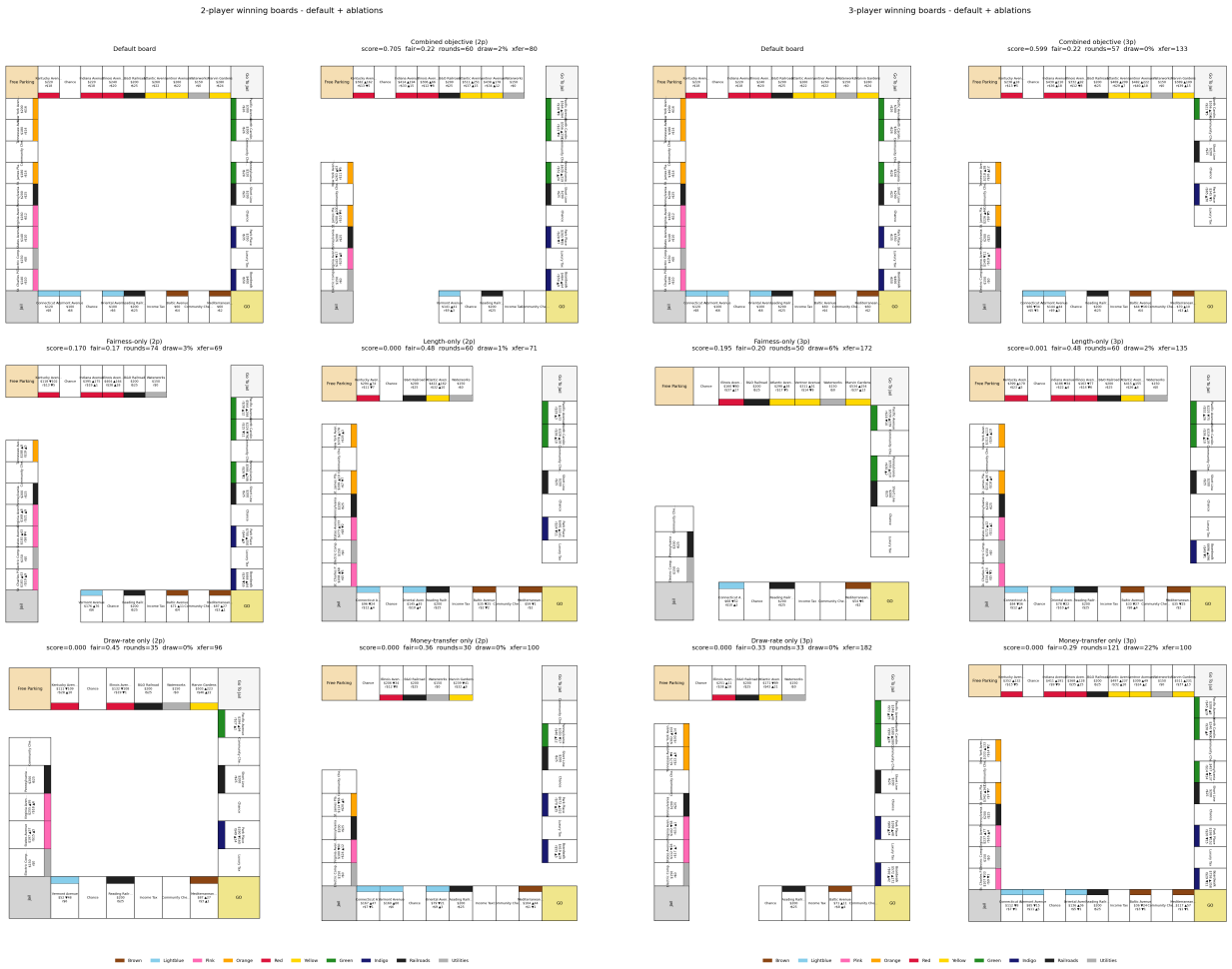
Design	training (n=200) ↓	deployment (n=1000) ↓	overfit ↓
GA-2p winner	0.705	0.773	+0.068
GA-3p winner	0.599	0.641	+0.042

Table 2: **Optimisation-set vs. deployment-set composite score, on each design's own training regime.** Header arrows: ↓ means lower is better. Both designs show small positive overfit (+0.068 and +0.042 respectively, both < 10% of the deployment score) relative to the headline ∼ 47% improvement over the default. Training number is the best score in the GA's own `evals.jsonl`; deployment number is the corresponding row of Table 1.

The Pareto frontiers in Figure 5 have visibly distinct shapes (different slope, different knee), so the cross-evaluation result respects the geometry of the design space rather than being seed noise.

4.4 Per-strategy heatmaps: *what* did the optimiser actually fix?

The composite score averages over only the 10 sampled evaluation matchups; it can move down even if a different subset of strategy pairs got worse. The full 30 × 30 strategy-vs-strategy win-rate matrix on the optimised board (Figure 6, left) and its difference against the default-board matrix (right) decompose what



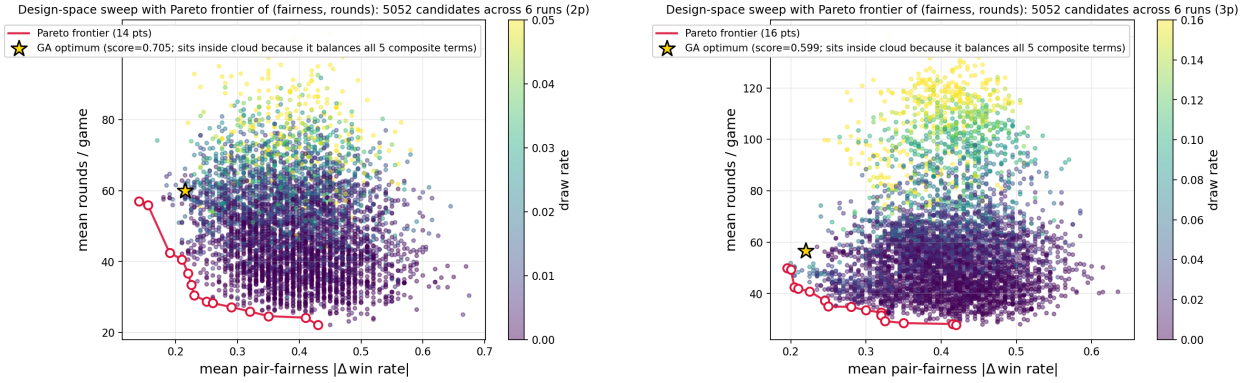
(a) 2 players (b) 3 players

Figure 4: **Each ablation produces a visibly different physical board.** Six panels per player count: the default board (top-left) and the winner of each of the five GA runs (combined, fair-only, length-only, draw-only, money-only). The combined winner balances all five composite terms; each ablation winner pushes its single term to its bound, often by aggressively pruning or duplicating colour-group properties. This is the same non-redundancy finding as Figure 3, made spatial.

the GA actually changed across all $\binom{30}{2} = 435$ unique strategy pairs in the pool.

The headline finding is that *board design moves system-level metrics aggressively, and per-pair strategy fairness much more modestly.* Mean $|W - 0.5|$ across the full 30×30 matrix drops from 0.21 (default) to 0.20 on the GA-2p winner ($\sim 5\%$ reduction), and from 0.26 to 0.20 on the GA-3p winner ($\sim 21\%$ reduction). Compared to that, the system-level metrics improve by much larger relative margins on the GA-2p board: game length falls from 103.9 to 62.2 rounds (a -40% drop), draw rate from 7.4% to 1.7% (-77%), inter-player money transfer rises from 49.7 to 73.3 \$/round (closer to the target of 100), and the mean fairness on the 10 evaluation matchups falls from 0.454 to 0.215 (-53%). Board design is a strong lever for pacing, decisiveness, and matchup-set fairness; it is a weak lever for population-wide skill ordering, which is structural to the strategy pool.

A related observation is that some matchups are immutable under any board configuration in the search space. The most asymmetric pair on the GA-2p winner is *Trader* vs. *RailroadKing* (100%/0%). No setting of cost multipliers, rent multipliers, or keep-mask bits eliminates this; it is intrinsic to a strategy that refuses to buy 22 of 28 ownable properties, and the GA cannot reach it through the design space. This is the substantive negative result of the section: board design is a lever for system-level properties (length, draws, interactivity)



(a) 2p

(b) 3p

Figure 5: **The (fairness, rounds) trade-off surface and its Pareto frontier.** Each marker is one of 5,052 candidates evaluated across all six search runs (random + GA + four single-objective ablations) on the 66-dim keep-mask design space; colour encodes draw rate. The crimson line traces the actual 2D Pareto frontier of (fairness, rounds), both minimised: 14 points at 2p, 16 at 3p, with a clean knee around fairness 0.22 and rounds $30-40$ at 2p (a slightly higher knee at 3p). The gold star is the combined-objective GA optimum; it sits *inside* the cloud rather than on the 2D frontier because the composite weights five terms (fairness, max-fairness, length, draw, transfer), not just two — the GA's optimum is on the 5D Pareto surface, of which the (fairness, rounds) projection is a shadow. The 2p sweep reaches mean rounds as low as ~ 22 (the keep-mask shrinks the lap aggressively), while the 3p frontier bottoms out closer to ~ 35 rounds; the two clouds have visibly different shapes, which is what makes the cross-evaluation finding (Table 1) a property of the design space rather than seed noise.

and for matchup-set fairness, but agent-level dominance on specific strategy pairs is structural to the strategy pool. Closing those residual gaps requires a strategy-side intervention (rule changes, matchmaking, or pool curation), not more search over the design space.

4.5 LLM cross-class agreement

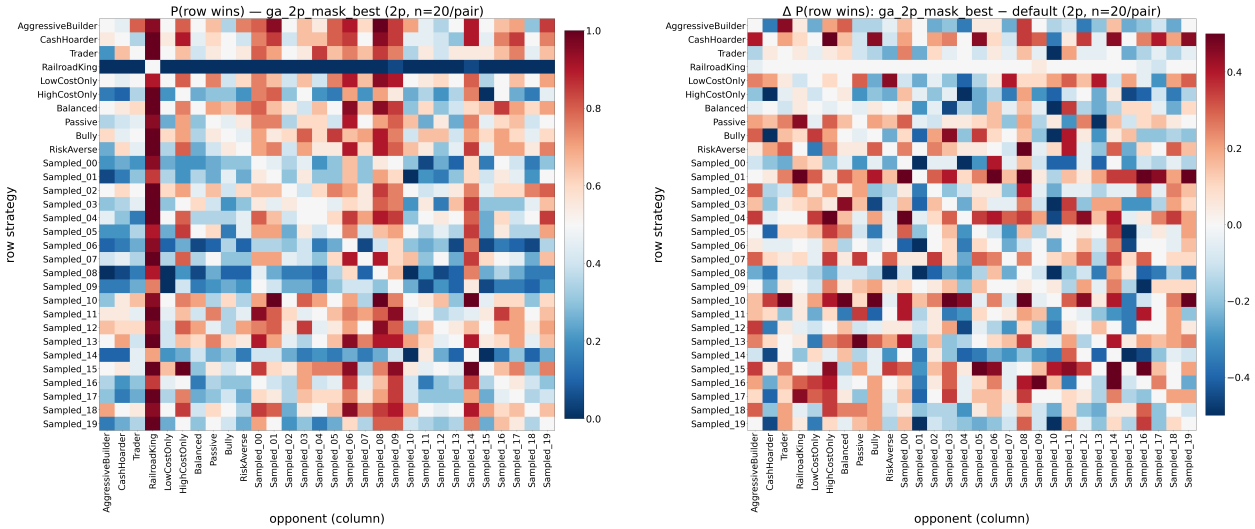
The cross-class probe re-ran 80 games on the optimised boards under all-LLM seats (Qwen2.5-1.5B-Instruct, greedy decoding) using the structured `STATE/ECHO/REASON/ANSWER` prompt. Per-decision logs record prompt, raw response, parse path, and per-field echo mismatches. Across 2,207 LLM calls, the v2 prompt produced zero model-side state hallucinations (Figure 7), at the cost of a small additional retry budget over the free-form v1 baseline.

The v1 prompt exposed a second-order finding worth reporting in its own right. v1's validator had a subtle integer/float type mismatch in the cash field, and retries against a wrong validator induced *more* plausible-looking confabulation rather than less, because the model was being told a correct value was wrong and produced a different "corrected" answer to satisfy the validator. We document this rather than tuning the validator until it disappears: a protocol whose failure modes are reported alongside its results is auditable; one whose are not is a benchmark trick.

4.6 LLM-driven GA and the fairness gap

We re-ran the GA with the LLM as evaluator (population 8, 5 generations, 5 seeds per candidate, 2-player only). The LLM-GA converges (Figure 10) and produces a winning board that beats the default under all-LLM evaluation. Cross-evaluating that board under the rule-based pool produces the diagnostic finding of the project (Figure 11): the LLM-GA winner is better than the default under either evaluator, but loses to the rule-based GA winner under the diverse pool. The fairness gap is 0.20 vs. 0.379 at 2p and 0.05 vs. 0.404 at 3p, and *widens* with more identical seats.

The interpretation is straightforward and is the headline diagnostic of the project: the LLM is good



(a) Optimum (2p)

(b) Optimum minus default (2p)

Figure 6: 30×30 **strategy-vs-strategy win-rate matrices. (a)** Win rates on the GA-2p winning board, P(row beats column); the diagonal is fixed at 0.5. **(b)** Difference against the default board, centred at zero. Across the full $\binom{30}{2} = 435$ unique pairs, mean $|W - 0.5|$ drops from 0.21 (default) to 0.20 on the GA-2p winner ($\sim 5\%$ reduction). The GA improved system-level metrics (length, draws, transfer) by much larger relative margins on the same board: board design is a stronger lever for those than for population-wide skill ordering.

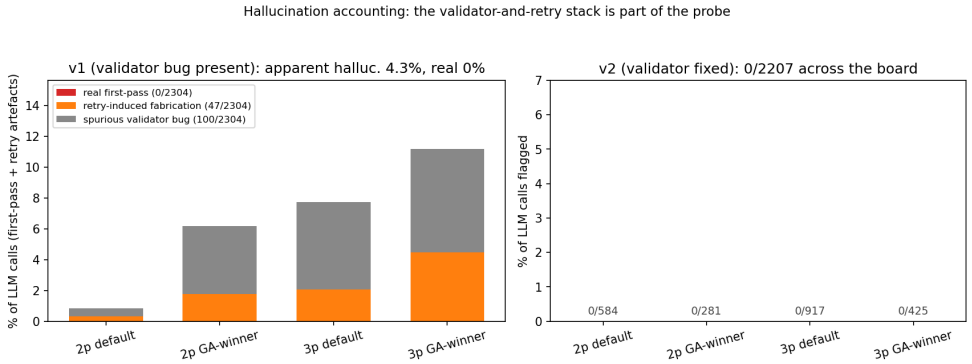


Figure 7: **The validator-and-retry stack is part of the probe: a faulty validator does not just produce noise, it pulls the model toward fabrication.** Per-call validation outcomes for the v1 prompt (left, with a known validator bug) and the v2 prompt (right, bug fixed). Each bar is stacked into three categories: *real first-pass* hallucinations (the model fabricated state on its first attempt), *retry-induced fabrication* (the model's first-pass output was correct but the buggy validator flagged it, and the model produced fabricated state on retry to "satisfy" the flag), and *spurious validator bug* (the validator falsely flagged correct outputs that the model did not change). On v1 across 2,304 calls, real first-pass = 0, retry-induced = 47, spurious = 100; the model itself never hallucinated. v2 drives all three categories to zero across all 2,207 calls. The reportable second-order finding: validating against a wrong ground truth is worse than not validating, because retries can teach the model to produce the "correct" wrong answer.

enough to drive an optimiser to a non-trivial design, but using a *single* agent personality as the evaluator collapses the worst-case fairness term. Evaluator diversity is therefore not optional for fairness-sensitive design. A single LLM or RL agent as evaluator is a benchmark; a strategy pool is a beta-testing surrogate.

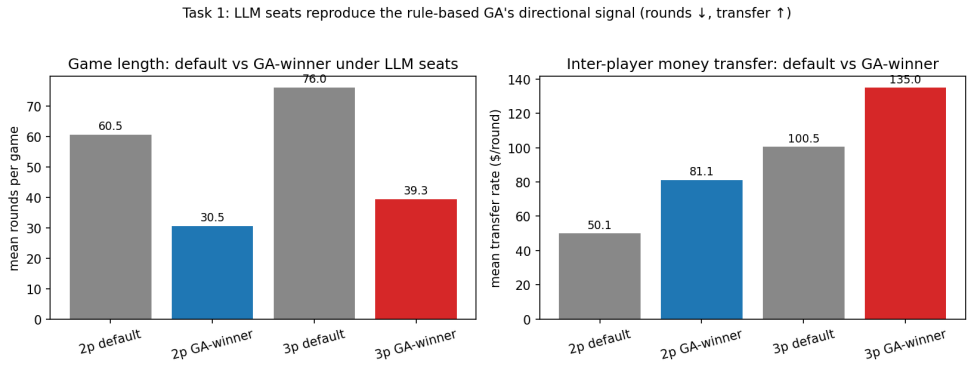


Figure 8: **LLM seats independently reproduce the rule-based GA's directional signal.** Mean rounds per game (left) and inter-player money transfer (right), measured under all-LLM seats on the default board and on the rule-based GA winner, at 2 and 3 players. The GA-winner board cuts rounds from 60.5 to 30.5 at 2p (−50%) and from 76.0 to 39.3 at 3p (−48%), and raises transfer from 50.1 to 81.1 $/round at 2p (+62%) and from 100.5 to 135.0 $/round at 3p (+34%). Both directional effects match what the rule-based 30-strategy pool found on the same boards (Section 4.1): two independent agent classes, run on the same designs, see the same change. Fairness is not included here because absolute fairness scores are not directly comparable across evaluators (a single-personality LLM evaluator has structural biases that the 30-strategy pool averages out, which is precisely what Section 4.6 measures).

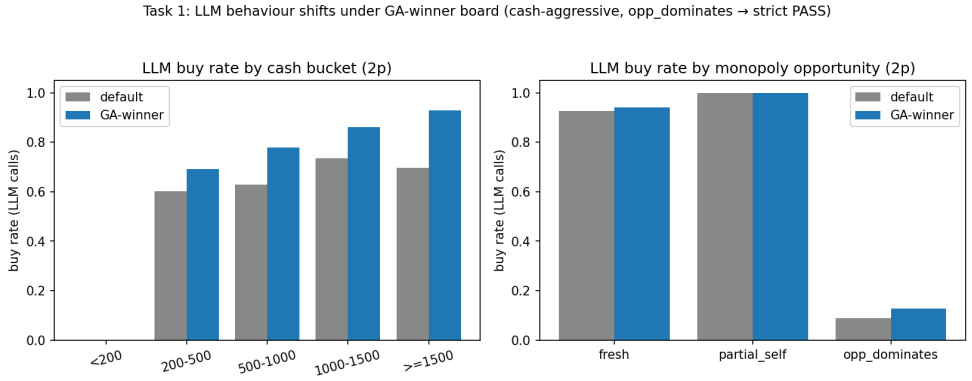
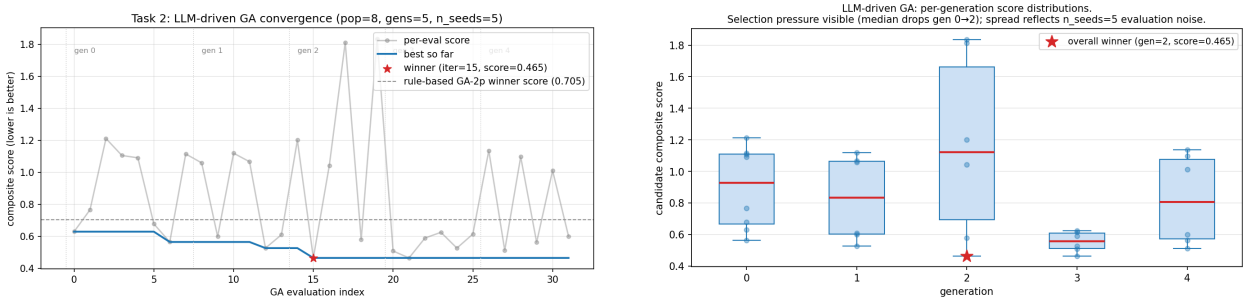


Figure 9: **LLM buy-rate slices, default board vs. GA-2p winner. (a) By cash bucket.** Buy rate conditioned on the LLM's available cash at decision time. The rate rises monotonically with cash, the rational ordering for any non-degenerate strategy; the empty < 200 bucket reflects the affordability pre-filter that short-circuits the LLM call when the player cannot buy anyway. On the GA-winner board the LLM commits its cash hoard more aggressively in the ≥ 1500 bucket (~ 0.93 vs. ~ 0.70 on the default), so the optimised board successfully pulls behaviour out of the cash-hoarding regime. **(b) By monopoly-completion situation.** *fresh*: no player owns any property in the landed colour group, so a purchase starts a new collection. *partial_self*: the LLM already owns one or more properties in the group and the opponent does not dominate it. *opp_dominates*: the opponent owns at least half the group and at least as many properties as the LLM, so buying mostly funds their next rent payment. Buy rates respond rationally across all three: ~ 1.00 on *partial_self* (saturated buy), ~ 0.94 on *fresh*, and ~ 0.13 on *opp_dominates* (strict PASS). The probe is not behaving as a uniform random oracle, and the directional responses are essentially identical on both boards.

4.7 Reproducibility check

Re-running any $(\theta, \text{matchups}, \text{seeds})$ triple yields byte-identical metrics. We verified this by re-running two randomly chosen JSONL lines from each major run and diffing the outputs. Every number in this report



(a) LLM-GA best-so-far

(b) Score distribution

Figure 10: **An LLM-only evaluator can drive the same GA loop. (a)** Per-evaluation composite score (grey) and best-so-far (blue) over the GA's 32 evaluations (population 8, 5 generations, n_{seeds}=5, with elitism keeping the unique evaluation count at 32). The dashed reference is the rule-based GA-2p winner's score under the same all-LLM evaluator (0.728): the LLM-driven loop finds a board (red star, iter 15, score 0.465) that beats even the rule-based winner when scored by an LLM. **(b)** Per-generation distribution of candidate scores; the median visibly drops from generation 0 to generation 2 (selection pressure), and the spread within each generation reflects the n_{seeds}=5 evaluation noise. Total wall time 4h 58min on a single 12 GB GPU. The LLM-driven loop converges; whether the converged board is a real design optimum or only an LLM-evaluator optimum is what Figure 11 resolves.

can be regenerated from a seed plus a JSONL line; the seed schedule is recorded in each run's `meta.json`.

4.8 Limitations

- **Strategy-pool coverage.** The 17-dim parametric space is broad but not exhaustive, and human players behave outside it. The 20 sampled strategies broaden the pool beyond the 10 archetypes and the sampling procedure is documented and reproducible, but a real deployment would want to learn the strategy distribution from logs.
- **No human playtest.** All metrics are internal to the agent loop. Whether either agent class's ranking actually predicts human play is untested, and confirming it would require a human playtest with multiple boards and many testers; we treat that as out of scope for this work. The falsifiability claim of the project is correspondingly weaker: we have shown that two agent classes agree directionally, not that either predicts human play.
- **LLM scale.** Qwen2.5-1.5B is the only LLM in the study. Larger models or different prompt styles may change the agreement profile. Greedy decoding does, however, make the probe deterministic.
- **Single-board scope.** All optimisation runs on the standard 40-cell US Monopoly layout (with structural shrinkage via keep-mask). Other game families would require a new design-space encoder, but the rest of the pipeline (objective, pool, GA, validator) carries over unchanged.

4.9 Lessons for multi-agent system design

1. **Strategy-pool diversity beats single oracle.** A 30-strategy parametric pool is a more honest evaluator than the strongest single agent we can train in the same time budget; the cross-evaluator gap (Section 4.6) is the empirical evidence.
2. **Robust aggregation belongs in the objective.** A worst-pair fairness term ($F_{\max}$) prevents the optimiser from improving the average at the expense of the tail; CVaR or a hard min generalise the same idea further.
3. **Per-objective ablations expose redundancy.** If two terms drive the same optimum, drop one; if they drive different optima (this work), keep both and report the trade-off explicitly.

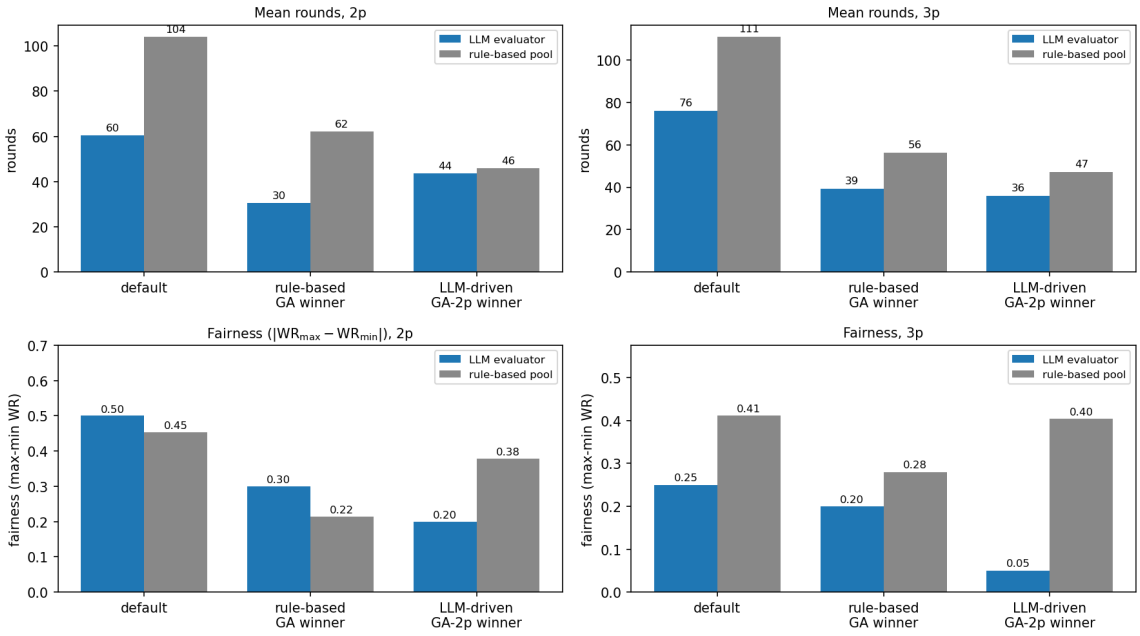


Figure 11: **Rounds-effect transfers across evaluators; fairness-effect does not.** A 2×2 grid scores three boards (default, rule-based GA winner, LLM-driven GA-2p winner) on two metrics (mean rounds, top row; fairness $|WR_{\max} - WR_{\min}|$, bottom row) under two evaluators (LLM seats, blue; 30-strategy rule-based pool, grey), at 2 and 3 players. The rounds-effect agrees in direction across evaluators on every board: a board that shortens games under one evaluator shortens them under the other. The fairness-effect, however, diverges sharply on the LLM-driven GA-2p winner: under the LLM evaluator it scores 0.20 at 2p and 0.05 at 3p (looks like the fairest board); under the rule-based pool it scores 0.38 at 2p and 0.40 at 3p (barely better than the default's 0.41 on the 3p panel, while the rule-based GA winner achieves 0.28). The fairness gap *widens* with player count: more identical seats produce more correlated mistakes for a single-personality evaluator, and the strategy pool's diversity is what surfaces them. This is the central empirical claim of the paper: evaluator diversity is not optional for fairness-sensitive design.

4. **Evaluation context is itself a design variable.** The 2p and 3p winners do not coincide; an optimisation result claimed under one regime should be re-evaluated under counterfactual conditions before it ships.
5. **Variance reduction beats more samples.** Shared random numbers across candidates made GA convergence visible at 100 games per candidate; uncorrelated evaluations would have required $\sim$ 1000 games for the same signal.
6. **The validator-and-retry stack is part of the probe.** v1's retry-induced fabrication is documented as a finding rather than patched away; a benchmark whose failure modes are not reported is a benchmark trick.

5 Team responsibilities

Selim Emir Can. Closed-loop optimisation pipeline (`ParametricPlayer`, strategy pool, design space, simulator wrapper, objectives, GA); LLM probe layer (`LLMPlayer`, structured `STATE/ECHO/REASON/ ANSWER` prompt, deterministic per-field validator, retry loop, per-decision logging); Task 1 (LLM-only evaluation) and Task 2 (LLM-driven GA); cross-evaluation harness; figure generation; report writing.

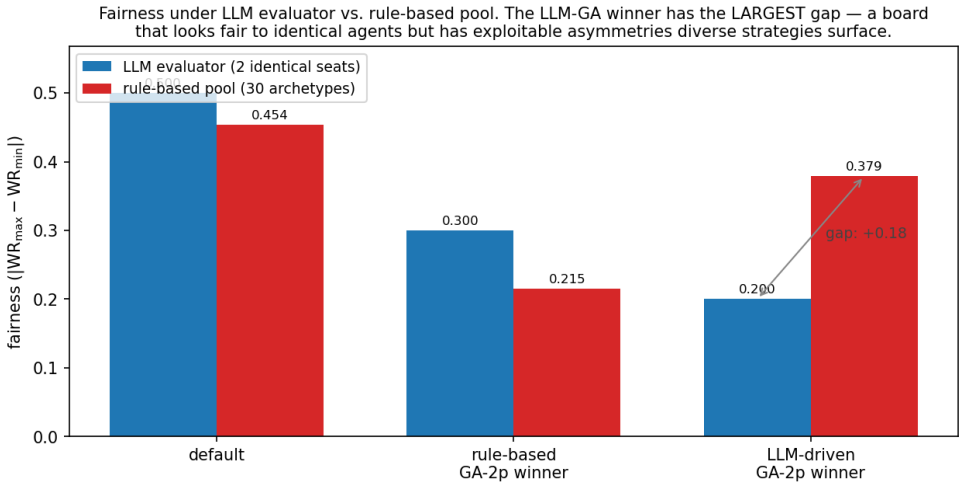


Figure 12: **The LLM-driven GA winner has the largest evaluator disagreement, and the sign of the gap flips.** Fairness ($|WR_{\max} - WR_{\min}|$, 2-player) on three boards (default, rule-based GA-2p winner, LLM-driven GA-2p winner) under two evaluators: the LLM with two identical seats (blue) and the 30-strategy rule-based pool (red). On the default and the rule-based GA winner, the LLM evaluator returns a *higher* fairness score than the rule-based pool (0.500 vs. 0.454 and 0.300 vs. 0.215): two identical LLM seats over-count asymmetry. On the LLM-driven GA-2p winner the sign *flips*: the LLM scores 0.200 but the rule-based pool scores 0.379, a gap of ~ 0.18 in the opposite direction. The board looks fair to identical agents while diverse strategies surface exploitable asymmetries the LLM evaluator cannot see. This is the same effect as the bottom-left panel of Figure 11, isolated and annotated.

Alaz Cig. [FILL IN: 1–2 sentences on Alaz's contributions. The git history on the `round1-phase1` and `main` branches contains parallel work I did not have visibility into; please replace this placeholder with the canonical breakdown before submission.]

A Implementation details

This appendix collects implementation details that are useful for reproducibility but would clutter the main text. All source paths are relative to `monopoly/` on the project repository.

A.1 LLMPlayer prompt and validator

Model and decoding. Qwen2.5-1.5B-Instruct [7] loaded with the `transformers` library, run with greedy decoding (`do_sample=False`), `max_new_tokens=160`, and additional EOS tokens `<|im_end|>` and `<|endoftext|>`. The same model and decoding settings are used for all four runs (LLM-only evaluation and the LLM-driven GA, at 2p and 3p).

Conversation structure. Each LLM buy decision is a six-message conversation: 1 system message, 4 few-shot user/assistant exchanges (2 *BUY* and 2 *PASS* demonstrations), and 1 runtime user message describing the actual decision. The full transcript with all four few-shot examples is checked into the repository at `prompts/llm_player_prompts.txt`.

System prompt (v2, verbatim).

> *You are a strategic Monopoly player who decides whether to buy properties. Your goal is to win by bankrupting opponents through rent. You must NOT mindlessly buy every property: passing is correct when (a) cash is dangerously low, (b) the property is in a group an opponent already dominates and you have no path to monopoly, or (c) the rent return is poor relative to the cost. Equally, buying is correct*

when the property advances a group you already own most of, or when it denies the group to an opponent who would otherwise complete it.

You receive a STATE block with 10 fields. Before reasoning, you MUST emit an ECHO block that copies all 10 STATE fields verbatim (numeric values must equal STATE; string values must equal STATE). The ECHO block is checked deterministically – if it differs from STATE in any field you will be asked to redo the answer. Only after the ECHO block do you write REASON and ANSWER. Do not invent any number or name not present in STATE. A purchase ONLY "completes a monopoly" when `you_own_in_group + 1 == group_size`*; do not claim completion otherwise.*

Reply in EXACTLY this format and order: `ECHO:` *followed by the 10 STATE fields (*`cash`*,* `property`*,* `group`*,* `cost`*,* `base_rent`*,* `you_own_total`*,* `you_own_in_group`*,* `group_size`*,* `opp_own_in_group`*,* `opp_own_total`*); then* `REASON: <one short sentence citing only STATE numbers>`*; then* `ANSWER: BUY` *or* `ANSWER: PASS`*. Do not output anything else.*

v1 vs. v2 prompts. The v1 prompt was free-form: it asked for "BUY" or "PASS" with a one-sentence reason but did not require the ECHO copy of STATE. The v1 validator parsed the free-form response with a regex and was prone to a now-fixed integer/float type mismatch on cash (Section 4.5). v2 added (i) the verbatim ECHO requirement, (ii) deterministic per-field equality checking on the echoed values, and (iii) a corrective retry loop that replays prior assistant turns plus a follow-up user message naming the mismatched fields. Both versions share the same four few-shot examples and the same STATE format.

Echo validator. `LLMPlayer._check_echo` parses the ECHO block with a multiline regex and equality-checks each of the 10 fields against ground-truth state, with a \$0.50 tolerance on cash to absorb float drift from rent multipliers (the original fixed-int validator caused 100 spurious flags in v1).

Retry mechanism. On any echo mismatch, the player retries up to `MAX_RETRIES = 4` times. Each retry replays prior assistant turns plus a corrective user message that names the mismatched fields. The final attempt's parsed answer is the decision used by the simulator, even if it still has echo issues; the per-decision JSONL log records every attempt for post-hoc analysis.

Pre-filter. Affordability and cash-floor checks short-circuit the LLM call when the decision is forced (cash $<$ cost forces PASS, etc.). This eliminates 60–80% of would-be calls and keeps the LLM evaluation pipeline within practical wall-clock budgets.

A.2 GA, ablations, and reproducibility

Search hyperparameters (canonical protocol, logs in `logs/optimizer_v3/`).

- GA: `--pop 30 --generations 30 --elitism 2`, tournament selection k=3, uniform crossover, Gaussian mutation σ=0.1 on continuous dims, 1-bit-flip mutation on the keep-mask. Total evaluations: $30 + 29 \times 28 = 842$.
- Random search: `--iters 842` for matched evaluation budget.
- Per-candidate evaluation: `--n-games 200`, distributed as 10 fixed matchups $\times$ 20 games per matchup, with seeds shared across all candidates.
- Composite weights $w = (1, 0.5, 0.5, 0.3, 0.3)$ for $(\overline{F}, F_{\max}, |\overline{R}-R^*|/R^*, \overline{D}, |\overline{T}-T^*|/T^*)$ with targets $R^* = 60$ rounds, $T^* = 100$ \$/round.
- Single-objective ablations set the chosen weight to 1 and the others to 0; same search hyperparameters otherwise.
- Cross-evaluation at n=1000 games per cell (deployment table) uses the same matchup seeds.

Reproducibility.

- All seeds (`--base-seed=42`, `--search-seed=0`, `--matchup-seed=1234`) are recorded in each run's `<run_name>.meta.json`.
- Re-running any `(design vector, matchups, seeds)` triple yields byte-identical metrics. We verified this by diffing two re-runs of the same config.
- Trigger scripts: `scripts/rerun_strategy_experiments_overnight.bat` (rule-based, ∼2.5h CPU) and `scripts/rerun_llm_phase_c_v3.bat` (LLM-only evaluation re-run, ∼6h on a 12 GB GPU).

B Mapping to the CS348K grading questions

The CS348K final-report guidelines ask two specific questions. We state both verbatim and answer each in one paragraph; the body of the report is structured around exactly these two questions, so this appendix is a quick reader guide.

(1) "What are the questions or goals your project aims to answer?" We ask whether agent simulations can substitute for human playtesting in game design under two qualifications: (a) the agents are imperfect and acknowledged as such, and (b) the unit of evidence is *cross-class agreement*: two independent agent classes (a 30-strategy parametric rule-based pool and a single-personality LLM with structured state validation) ranking the same candidate boards in the same direction. The falsifiable hypothesis is that the directional ranking under one class matches the other on the optimised boards, with the LLM's per-decision hallucination rate below 1%. A stronger validation path (human playtest with multiple boards and many testers) is out of scope for this work, treated as a known limitation (Section 4.8). The full statement is in Section 1.

(2) "What experiments should be done to answer that question, and how will you know from the outcome of the experiment that you have succeeded?" Seven experiments, each with an explicit success criterion:

1. Default-board baseline at n=1000 (Section 4.1). *Success:* deterministic scores with 95% Wilson intervals on every metric.
2. GA vs. random search at matched ∼842-evaluation budget (Section 4.1). *Success:* GA crosses the default reference within budget, beats random by a non-trivial margin, and is bit-reproducible. *Outcome:* GA wins by 13% in both player counts.
3. Single-objective ablations (Section 4.2). *Success:* visibly different per-aspect optima, confirming the composite is non-redundant. *Outcome:* confirmed; see Figures 3 and 4.
4. $2p \leftrightarrow 3p$ cross-evaluation at n=1000 (Section 4.3). *Success:* improvement over default $\geq 20\%$ in the off-regime; quantify the specialisation gap. *Outcome:* both designs improve over default by $\sim 47\%$ in the training regime; off-regime gap is $16-24\%$ on the composite.
5. Per-strategy heatmap (Section 4.4). *Success:* reduction in the 30×30 mean $|W-0.5|$, with the systematic pattern of which matchups are immutable surfaced honestly. *Outcome:* 5% reduction at 2p, 21% at 3p; the *Trader vs. RailroadKing* 100/0 pair is genuinely immutable under any board configuration.
6. LLM cross-class probe (Section 4.5). *Success:* rounds and transfer move in the same direction the rule-based pool predicted on the same boards; per-decision hallucination rate below 1%. *Outcome:* 0/2,207 first-pass hallucinations under v2 ECHO validation; rounds and transfer effects strengthen on the GA-winner board (-50% rounds at 2p).
7. LLM-driven GA + cross-evaluator gap (Section 4.6). *Success:* the LLM-driven loop converges to a non-trivial design; cross-evaluating it under the rule-based pool exposes (or fails to expose) the single-personality blind spot. *Outcome:* the cross-evaluator fairness gap is +0.18 at 2p and +0.35 at 3p, widening with player count. This is the central diagnostic claim of the paper.

References

[1] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.

[2] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.

[3] Christoffer Holmgård, Antonios Liapis, Julian Togelius, and Georgios N. Yannakakis. Evolving personas for player decision modeling. In *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, 2014.

[4] Averill M. Law and W. David Kelton. *Simulation Modeling and Analysis*. McGraw-Hill, 5th edition, 2014. Chapter on Common Random Numbers (CRN) for variance reduction in stochastic simulation.

[5] Antonios Liapis, Georgios N. Yannakakis, and Julian Togelius. Sentient sketchbook: Computer-aided game level authoring. In *Proceedings of the 8th International Conference on the Foundations of Digital Games (FDG)*, pages 213–220, 2013.

[6] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, pages 1–22, 2023.

[7] Qwen Team. Qwen2.5 technical report. Technical report, Alibaba Group, 2024.

[8] Noor Shaker, Julian Togelius, and Mark J. Nelson. *Procedural Content Generation in Games*. Computational Synthesis and Creative Systems. Springer, 2016.

[9] J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S. Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, Niall Williams, Yashas Lokesh, and Praveen Ravi. Pettingzoo: Gym for multi-agent reinforcement learning. *Advances in Neural Information Processing Systems*, 34, 2021.

[10] Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.

[11] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.